
GRAFIKA KOMPUTEROWA LAB_2

Table of Contents

1. CEL CWICZENIA	1
2. ZADANIA	1
2.1.a Binarizacja obrazu monochromatycznego z ustalonym progiem.	1
2.1.b Binarizacja obrazu monochromatycznego z progiem wyznaczonym metodą Otsu.	3
2.1.c Adaptacyjna (lokalna) binaryzacja obrazu monochromatycznego.	5
2.2. Uśrednianie obrazu monochromatycznego operatorami o dowolnych prostokątnych rozmiarach.	6
2.3 Wyznaczania krawędzi w obrazach monochromatycznych za pomocą operatorów Sobela, Prewitta i Roberts.	9
2.4 Wyznaczanie liczby Eulera w obrazie binarnym.	17
2.5 Przetwarzanie obrazu monochromatycznego za pomocą zdefiniowanego operatora 3x3.	18
Typowy koniec skryptu	20

Joanna Dagil 231008

1. CEL CWICZENIA

Zapoznanie się z punktowymi i lokalnymi metodami przetwarzania obrazów.

2. ZADANIA

2.1.a Binarizacja obrazu monochromatycznego z ustalonym progiem.

% Dla każdego pixela, jeżeli jego jasność jest większa niż zadany próg, to ustawiamy jego wartość na 255, w przeciwnym wypadku pozostawiamy wartość 0.

```
function Res = greying(name)
    Im = imread(name);
    [~, ~, d] = size(Im);
    if d > 1
        Res = rgb2gray(Im);
    else
        Res = Im;
    end
end
```

```
function Res = zad1A(Im, Th)

    [y, x] = size(Im);

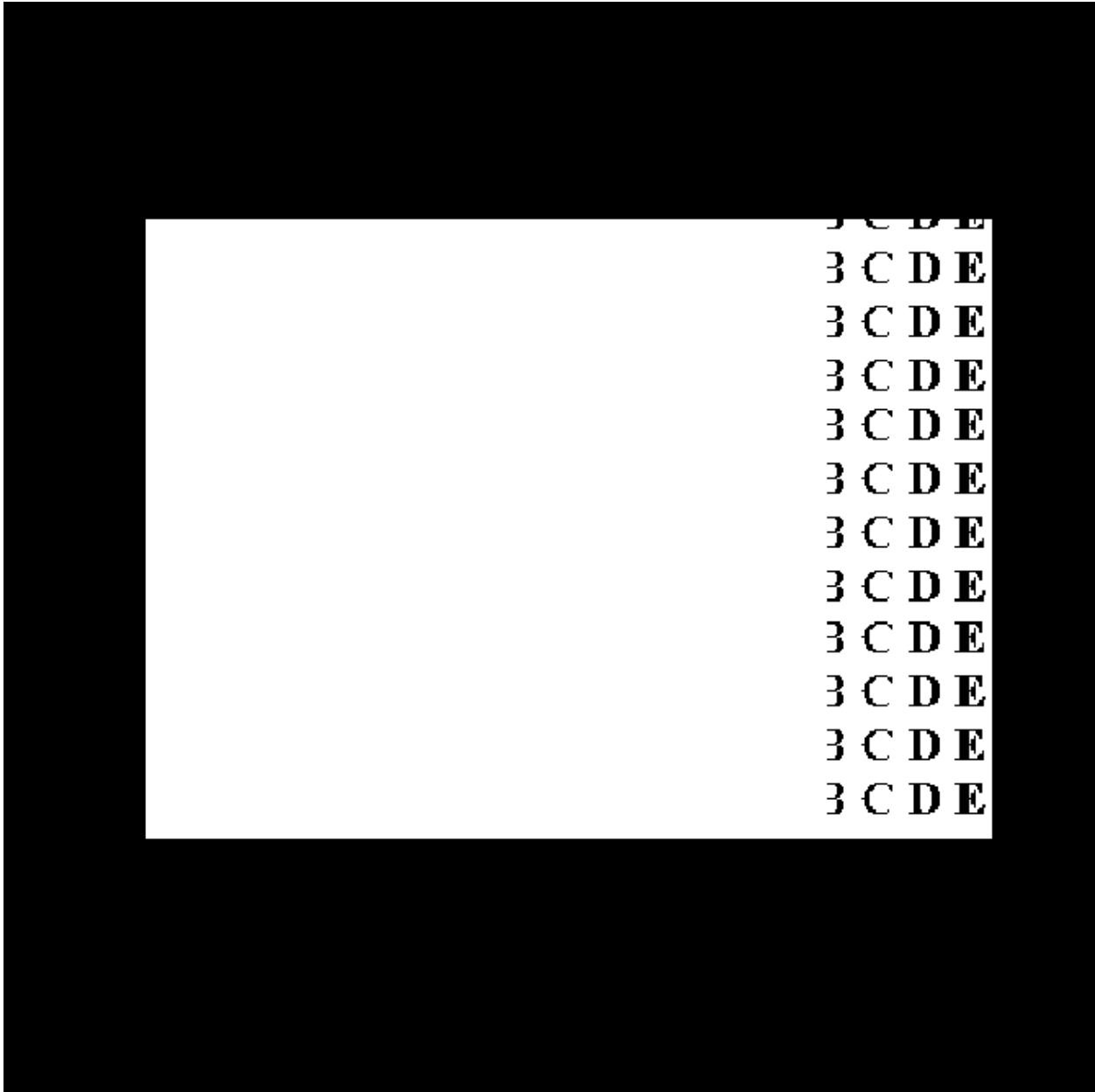
    Res = uint8(zeros(y, x));

    for m = 1:y
```

```
    for n = 1:x
        if Im(m,n) > Th
            Res(m,n) = 255;
        end
    end
end
end
end

Im1A = zad1A(greying('text1.bmp'),50);
figure(1); imshow(Im1A);
```





2.1.b Binaryzacja obrazu monochromatycznego z progiem wyznaczonym metodą Otsu.

% Metoda Otsu z standardowych zasobów Matlaba.

% Metoda Otsu wyznacza optymalny próg binaryzacji, który minimalizuje odchylenie standardowe jasności wśród pixeli sklasyfikowanych jako białe oraz wśród tych zakwalifikowanych jako czarne.

function Res = zad1B(Im)

[counts, ~] = imhist(Im);

```
Th = otsuthresh(counts);  
Res = imbinarize(Im, Th);  
  
end  
  
Im1B = zad1B(greying('text1.bmp'));  
figure(2); imshow(Im1B);
```



2.1.c Adaptacyjna (lokalna) binaryzacja obrazu monochromatycznego.

% Dla każdego pixela, próg binaryzacji wyznaczamy jako średnią z otoczenia 21x21 tego pixela. Jeżeli pixel leży zbyt blisko którejś krawędzi obrazu, rozmiar otoczenia powinien być odpowiednio mniejszy (pomijamy pixele, które wychodziłyby poza granice obrazu).

```
function Im1 = zad1C(Im)
    [y, x] = size(Im);

    dIm = double(Im);

    Im1 = uint8(zeros(y, x));

    for m = 1:y
        for n = 1:x

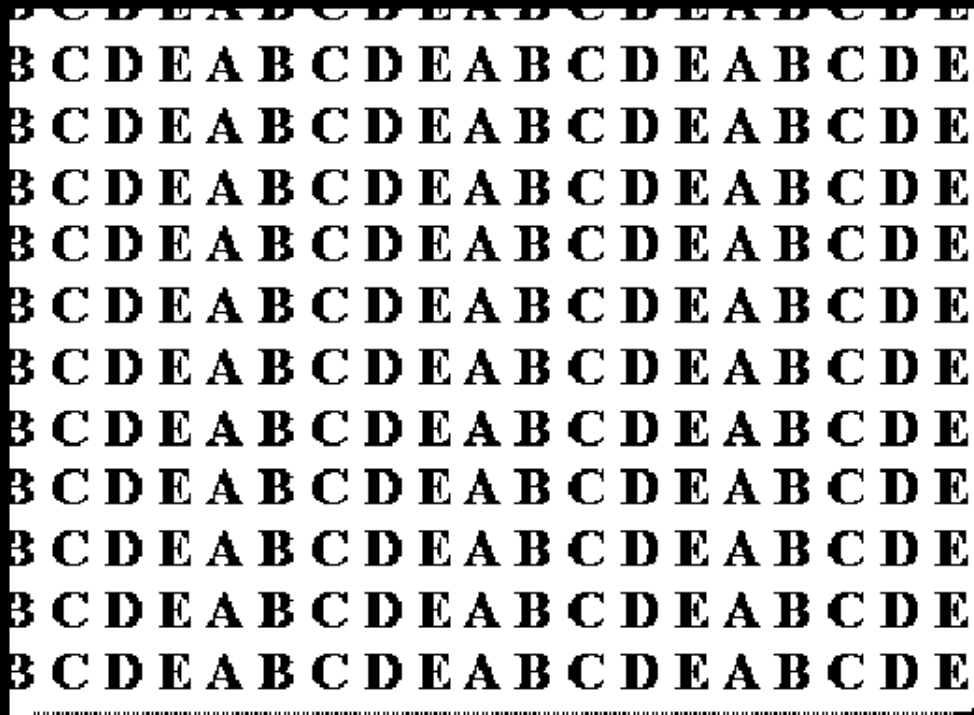
            ymin = max(1, m-10); ymax = min(y, m+10);
            xmin = max(1, n-10); xmax = min(x, n+10);

            no = (1 + ymax - ymin) * (1 + xmax - xmin);

            suma = sum(sum(dIm(ymin:ymax, xmin:xmax)));
            thr = suma / no;

            if dIm(m,n) > thr
                Im1(m,n) = 255;
            end
        end
    end

    Im1C = zad1C(greying('text1.bmp'));
    figure(3); imshow(Im1C);
```



2.2. Uśrednianie obrazu monochromatycznego operatorami o dowolnych prostokątnych rozmiarach.

```
function Res = zad2(Im, M, N)

    OP = ones(M, N) / (M * N);

    dIm = double(Im);

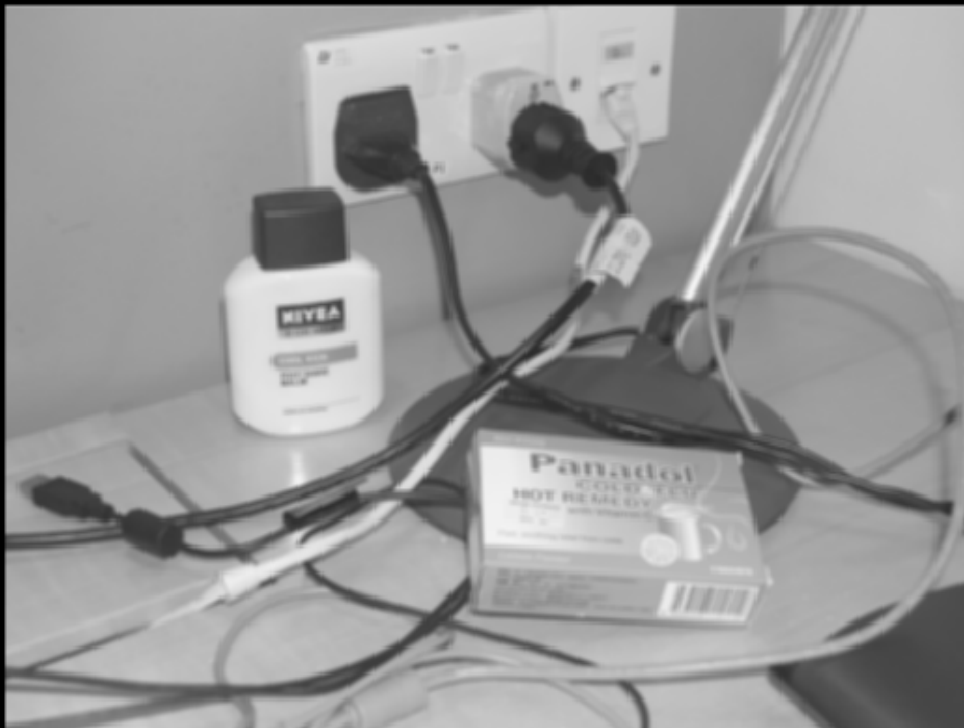
    Res = uint8(filter2(OP, dIm, 'same'));
```

end

```
Im2 = zad2(greying('A.bmp'),3,3);  
figure(4); imshow(Im2);
```

```
Im2 = zad2(greying('B.bmp'),7,7);  
figure(5); imshow(Im2);
```

```
Im2 = zad2(greying('C.bmp'),11,11);  
figure(6); imshow(Im2);
```







2.3 Wyznaczania krawędzi w obrazach monochromatycznych za pomocą operatorów Sobela, Prewitta i Robertsa.

```
function zad3(Im, N, H, V)

    dIm = double(Im);

    % gradienty
    ImH = abs(filter2(H, dIm, 'same'));
    ImV = abs(filter2(V, dIm, 'same'));
```

```
ImTot = ImH + ImV;

% normalizacja gradientow
ImH = 255 * ImH / max(ImH(:));
ImV = 255 * ImV / max(ImV(:));
ImTot = 255 * ImTot / max(ImTot(:));

Im1 = uint8(ImH);
Im2 = uint8(ImV);
Im3 = uint8(ImTot);

figure(N); imshow(Im1);
figure(N+1); imshow(Im2);
figure(N+2); imshow(Im3);
end

% Macierze Sobela.

SobelH = [1, 2, 1;
          0, 0, 0;
          -1, -2, -1];
SobelV = [1, 0, -1;
          2, 0, -2;
          1, 0, -1];

% Macierze Prewitta.

PrewittH = [1, 1, 1;
            0, 0, 0;
            -1, -1, -1];
PrewittV = [1, 0, -1;
            1, 0, -1;
            1, 0, -1];

zad3(greying('C.bmp'), 7, SobelH, SobelV);
zad3(greying('C.bmp'), 10, PrewittH, PrewittV);
```















2.4 Wyznaczanie liczby Eulera w obrazie binarnym.

% Posługujemy się wzorem: $E = S1 - S2 + S3$, gdzie:
% S1 - liczba czarnych pixeli,
% S2 - liczba par sąsiednich czarnych pixeli (w poziomie i w pionie),
% S3 - liczba czwórek sąsiednich czarnych pixeli (2x2).

```
function e = zad4(name)
```

```
    Im = imread(name);
```

```
S1 = 0;
S2 = 0;
S3 = 0;

[y, x] = size(Im);

for i = 1:y
    for j = 1:x
        if Im(i,j) == 0
            S1 = S1 + 1;

            if i < y && Im(i+1,j) == 0
                S2 = S2 + 1;
            end

            if j < x && Im(i,j+1) == 0
                S2 = S2 + 1;
            end

            if i < y && j < x && Im(i+1,j) == 0 && Im(i,j+1) == 0
                S3 = S3 + 1;
            end
        end
    end
end

e = S1 - S2 + S3;
end

e1 = zad4('binary1.bmp');
e3 = zad4('binary3.bmp');
```

2.5 Przetwarzanie obrazu monochromatycznego za pomocą zdefiniowanego operatora 3x3.

```
function Res = zad5(Im, OP)

    dIm = double(Im);

    dIm1 = abs(filter2(OP, dIm, 'same'));
    dIm1 = 255 * dIm1 / max(dIm1(:));

    Res = uint8(dIm1);
end

OP = -1 * ones(3,3) / 9;
OP(2,2) = 8 / 9;

Im5 = zad5(greying('C.bmp'), OP);
figure(13); imshow(Im5);

% Ten operator wykrywa piksela w obrazie, które otoczone są pikselami o
```

przeciwnych wartościach, a więc między innymi krawędzie z każdym kątem nachylenia. Wartość $8/9$ w środku maski wzmacnia piksel centralny, podczas gdy wartości $-1/9$ wokół niego osłabiają sąsiednie piksele.





Typowy koniec skryptu

```
% clear all; % usuniecie wszystkich zmiennych  
% close all; % zamkniecie wszystkich wykresow  
% clc % zresetowanie okna komend
```

Published with MATLAB® R2025b